

The Neuron: Introduction and Core Concepts

A **Neuron** (also called a **perceptron** or **node**) is the fundamental computational unit of a neural network, inspired by the structure of biological neurons. ☺

Structure and Function

A single neuron performs two key operations:

1. **Weighted Sum:** It takes multiple inputs, multiplies each input by an associated **weight**, and sums the results. A **bias** term is then added to this sum. This forms the **pre-activation** or **net input** (z). ☺ ☺

$$z = \left(\sum_{i=1}^n w_i x_i \right) + b$$

Where:

- x_i : input features.
 - w_i : weights (parameters learned during training). ☺
 - b : bias term (an offset parameter). ☺
2. **Activation:** The weighted sum (z) is passed through a non-linear **activation function** (f), which determines the neuron's output ($\hat{y}$). ☺

$$\hat{y} = f(z)$$

The non-linearity introduced by the activation function (like ReLU, Sigmoid, or Tanh) allows neural networks to model complex, non-linear relationships in data, which linear models cannot do. ☺

Fitting a Line (Linear Regression Analogy)

The process of **fitting a line** using linear regression provides the simplest analogy for how a neuron works before the non-linear activation.

- **Goal:** To find the best straight line ($y = mx + c$) that describes the relationship between an input feature (x) and an output target (y).
- **Neuron Mapping:**
 - **Input** x maps to the **feature** x_1 .
 - **Slope** m maps to the **weight** w_1 .
 - **Y-intercept** c maps to the **bias** b .
 - **Output** $\hat{y}$ maps to the **pre-activation** z .
- **The Fit:** The model is "fit" (trained) by adjusting the weight (w) and bias (b) to **minimize the distance** (error) between the predicted line ($\hat{y}$) and the actual data points (y). This error is quantified by a **Loss Function** (e.g., Mean Squared Error).

In this simple linear case, the model learns by adjusting two parameters: w and b .

🤔 How Does a Model Learn?

The learning process in any statistical or neural model—from fitting a simple line to training a large language model—is fundamentally the same: **optimization**.

The Three Steps of Learning

1. **Define the Loss Function (Cost Function):** This mathematically quantifies the "badness" of the model's current predictions. If the prediction $\hat{y}$ is far from the true target y , the loss is high. Examples: **Mean Squared Error (MSE)** for regression, **Cross-Entropy** for classification.
2. **Optimization Algorithm (Gradient Descent):** The goal is to find the set of weights ($\mathbf{W}$) and biases ($\mathbf{B}$) that minimize the Loss function $L(\mathbf{W}, \mathbf{B})$.
 - **Gradient:** The optimizer calculates the **gradient** (the partial derivative of the loss function with respect to each parameter). The gradient points in the direction of **steepest increase** in loss.
 - **Descent:** The optimizer updates the parameters by taking small steps in the **opposite direction** of the gradient (the direction of steepest decrease in loss).

$$\mathbf{W}_{\text{new}} = \mathbf{W}_{\text{old}} - \eta \frac{\partial L}{\partial \mathbf{W}_{\text{old}}}$$

Where η is the **learning rate**, controlling the size of the steps.

3. **Backpropagation:** In neural networks (multiple connected neurons), **backpropagation** is the algorithm used to efficiently compute the gradient of the loss with respect to every weight in the network, propagating the error backward from the output layer to the input layer.



Classification Code Preparation and Text Classification in Tensorflow

Classification Code Preparation (General Steps)

Before implementing a classifier in a framework like TensorFlow, several preparatory steps are essential:

1. **Data Preprocessing:** Clean and normalize the data. For **Text Classification**, this involves: 
 - **Tokenization:** Breaking text into words/subwords.
 - **Vocabulary Creation:** Mapping unique tokens to integers. 
 - **Padding/Truncation:** Making all sequences the same length. 
2. **Vectorization:** Convert text data into a numerical format (e.g., one-hot vectors, or more commonly, dense **Embeddings**). 
3. **Model Architecture:** Define the layers of the neural network (input layer, hidden layers with neurons and activation functions, output layer). 
4. **Compilation:** Specify the optimizer (e.g., Adam), the loss function (e.g., `categorical_crossentropy`), and the metrics (e.g., accuracy).

Text Classification in TensorFlow (High-Level)

TensorFlow (and its API, Keras) simplifies the implementation of text classification using a sequence of layers: [↗](#)

1. **Input Layer:** Takes the preprocessed integer-encoded sequences.
2. **Embedding Layer:** This is the first layer for text. It maps each integer index to a dense vector (word embedding). This layer learns semantic relationships.
3. **Recurrent/Convolutional Layer (Optional but Common):** Layers like **LSTM (Recurrent)** or **Conv1D (Convolutional)** process the sequence of embeddings to capture temporal or local patterns. [↗](#)
4. **Global Pooling/Flattening:** Condenses the output of the sequence layer into a fixed-size vector.
5. **Dense Layers (The Neurons):** One or more fully connected layers of neurons process the condensed feature vector. [↗](#)
6. **Output Layer:** A final dense layer with an activation function appropriate for the task: [↗](#)
 - **Sigmoid** for binary classification. [↗](#)
 - **Softmax** for multi-class classification. [↗](#)

The entire network, composed of these layers of interconnected neurons, is trained using the **Backpropagation** method described above to adjust all the weights in the embedding, recurrent, and dense layers simultaneously.

[↗ Sources](#)

Text Preprocessing: Code Preparation

Text Preprocessing transforms raw text into a numerical format suitable for machine learning models. The preparation involves tokenization, cleaning, and numerical mapping.

1. Cleaning and Tokenization

Before building the vocabulary, the text needs to be standardized.

Step	Description	Python Tools
Lowercasing	Converts all characters to lowercase to treat "The" and "the" as the same word.	<code>.lower()</code> method
Punctuation Removal	Eliminates punctuation marks that don't add semantic value.	<code>string.punctuation</code> and list comprehensions
Tokenization	Splits the text into meaningful units (words or sub-words).	<code>nltk.word_tokenize</code> or simple <code>.split()</code>
Stop Word Removal	Removes common, non-informative words (e.g., "a," "the," "is").	<code>nltk.corpus.stopwords</code>

2. Vocabulary Mapping

The final output of preprocessing is an integer representation.

Step	Description	Outcome
Vocabulary Creation	Create a list of all unique tokens remaining after cleaning.	<code>['word', 'dog', 'run', 'fast', ...]</code>
Word-to-Index (WTI)	Create a dictionary mapping each unique word to a unique integer ID.	<code>{'word': 1, 'dog': 2, 'run': 3, ...}</code>
Sequence Conversion	Convert the original tokenized text into a list of integer IDs.	<code>[1, 2, 3, 4, 1, ...]</code>

Text Preprocessing in TensorFlow

TensorFlow and Keras offer the `tf.keras.preprocessing.text.Tokenizer` class, which automates vocabulary creation and sequence conversion.

Python



```
import tensorflow as tf
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences

# 1. Initialize Tokenizer
# num_words limits the vocabulary size; oov_token handles out-of-vocabulary words
tokenizer = Tokenizer(num_words=10000, oov_token="<unk>")

# 2. Fit on Text (Build Vocabulary)
corpus = ["The dog ran fast.", "A fast car sped past."]
tokenizer.fit_on_texts(corpus)

# 3. Convert Text to Sequences
integer_sequences = tokenizer.texts_to_sequences(corpus)
# Example output: [[1, 2, 3], [4, 5, 6, 7]]

# 4. Padding/Truncation
# Ensures all input sequences have the same length (required by neural networks)
max_len = 10
padded_sequences = pad_sequences(
    integer_sequences,
    maxlen=max_len,
    padding='post' # or 'pre'
)
# Example output (assuming padding='post'): [[1, 2, 3, 0, 0, ...], [4, 5, 6, 7, 0, ...]]
```


Embeddings are dense, low-dimensional vector representations of discrete variables (like words). Instead of a sparse one-hot vector (which is huge and inefficient), a word is mapped to a vector of much smaller size (e.g., 50 to 300 dimensions).

- **Semantic Meaning:** Embeddings are designed such that words with similar meanings are located **close together** in the vector space (measured by cosine similarity).
- **Learning:** Embeddings can be pre-trained on a massive external corpus (like Word2Vec, GloVe) or learned directly during the training of a neural network using a `tf.keras.layers.Embedding` layer.

CBOW (Continuous Bag of Words)

CBOW is one of the original and most famous algorithms (part of the **Word2Vec** framework) designed to **learn word embeddings**.

The Task

CBOW's objective is to **predict a target word** given its surrounding context words (its "bag" of words).

- **Training Data Preparation:** The corpus is scanned to create training pairs (Context, Target). For a given **window size** (e.g., 2), the context consists of the 2 words before and the 2 words after the target word.
 - Example: Sentence: "The quick **brown** fox jumps" (Window=2)
 - Target word: **brown**
 - Context words: The, quick, fox, jumps
- **Architecture:** It uses a simple neural network with no hidden non-linearity, focused on learning the relationship between context and target.

Mechanism and Learning

1. **Input:** The context words are passed into the model (represented by their one-hot vectors).
2. **Projection:** The input vectors are multiplied by the initial **Input Weight Matrix** (W) to get their initial **word embeddings**.
3. **Averaging:** The context word embeddings are **averaged** together. This average vector represents the predicted topic or concept of the context.
4. **Prediction:** The averaged vector is multiplied by a second **Output Weight Matrix** (W') to produce scores for all words in the vocabulary.
5. **Softmax:** A Softmax function converts these scores into a probability distribution over the vocabulary.
6. **Learning:** The model adjusts the W and W' matrices using backpropagation to maximize the probability of the actual target word. **The final learned columns of the W matrix are the word embeddings.**

🐍 CBOW in TensorFlow

Implementing CBOW in TensorFlow often involves creating a **custom data generator** to prepare the (Context, Target) pairs, and then defining a simple Keras model.

Model Architecture

This model uses the Keras `Embedding` layer to hold the learned W matrix.

Python



```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, Lambda, Dense, Flatten
from tensorflow.keras import backend as K

vocab_size = 10000
embedding_dim = 100
window_size = 2 # Context words = 2 * window_size

# 1. Define the Keras Model for CBOW
def create_cbow_model(vocab_size, embedding_dim, context_size):
    model = Sequential()

    # Embedding Layer: Learns the input weight matrix (W)
    # The input shape is the number of context words
    model.add(Embedding(
        input_dim=vocab_size,
        output_dim=embedding_dim,
        input_length=context_size
    ))

    # Lambda Layer: Averages the context word embeddings (the core of CBOW)
    # The axis=1 averages across the word dimension
    model.add(Lambda(lambda x: K.mean(x, axis=1), output_shape=(embedding_dim,)))

    # Output Layer: Multiplies by the output weight matrix (W')
    # Softmax output size is vocab_size (probability for every word)
    model.add(Dense(vocab_size, activation='softmax'))
```

Python



```
# 1. Define the Keras Model for CBOW
def create_cbow_model(vocab_size, embedding_dim, context_size):
    model = Sequential()

    # Embedding Layer: Learns the input weight matrix (W)
    # The input shape is the number of context words
    model.add(Embedding(
        input_dim=vocab_size,
        output_dim=embedding_dim,
        input_length=context_size
    ))

    # Lambda Layer: Averages the context word embeddings (the core of CBOW)
    # The axis=1 averages across the word dimension
    model.add(Lambda(lambda x: K.mean(x, axis=1), output_shape=(embedding_dim,)))

    # Output Layer: Multiplies by the output weight matrix (W')
    # Softmax output size is vocab_size (probability for every word)
    model.add(Dense(vocab_size, activation='softmax'))

    # Compile the model
    model.compile(
        optimizer='adam',
        loss='categorical_crossentropy', # Used for predicting a single class (the target)
        metrics=['accuracy']
    )
    return model

# context_size is 2 * window_size, e.g., 4
cbow_model = create_cbow_model(vocab_size, embedding_dim, 4)
# cbow_model.summary()

# NOTE: The actual training requires feeding the model a large dataset of (Context, Target)
# where Context is a padded sequence of context IDs, and Target is a one-hot vector
# of the target word.
```
